3.3 Descriptive statistics

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 14 October 2026, 20:30

This handout provides a comprehensive, hands-on introduction to descriptive and bivariate statistics for both categorical and continuous linguistic data, including data visualisation, summary measures, correlation, and exporting tables, with practical exercises in R using real-world datasets.

Table of contents

Recommended reading . . . 2
Preparation . . . 2

Describing categorical data **3**
Frequency tables (one variable) . . . 3
Frequency tables ($\geq$ 2 variables) . . . 4
Percentage tables . . . 5
Plotting categorical data . . . 6
One variable . . . 6
Base R . . . 6
ggplot2 . . . 7
Two variables . . . 8
Base R . . . 8
Base R (fully customised) . . . 9
ggplot2 . . . 9
ggplot2 (fully customised) . . . 10
Exporting tables to MS Word . . . 15

Describing continuous data **17**
Measures of central tendency . . . 17
The sample mean . . . 17
Histogram (ggplot2) . . . 18
Density plot (ggplot2) . . . 19

Histogram (Base R) . . . 20
Density plot (Base R) . . . 21
The median . . . 21
Histogram (ggplot2) . . . 21
Density plot (ggplot2) . . . 22
Histogram (Base R) . . . 23
Density plot (Base R) . . . 24
Sample variance and standard deviation . . . 24
Quantiles . . . 27
Quartiles and boxplots . . . 28
Bivariate statistics . . . 29
Covariance . . . 29
Correlation . . . 30

Exercises **31**
Tier 1 . . . 31
Tier 2 . . . 31
Tier 3 . . . 35
References . . . 36

Script

You can find the full R script associated with this unit here.

Recommended reading

Theoretical introduction:

Baguley (2012, chap. 1 & 6)

Heumann et al. (2022: Chapter 3)

Application in R:

Wickham et al. (2023: Chapter 1)

Style guide:

e.g., APA Numbers and Statistics Guide (7th ed.)

Preparation

This unit draws on the genitive alternation data compiled by Grafmiller (2023) and previously used for publication in Grafmiller (2014).

```
# Libraries
library(tidyverse) # for fancy plots and comfortable data manipulation
# For publication-ready tables
library(crosstable)
library(flextable)
%>%
# Load data from working directory
genitive <- read.csv("Grafmiller_genitive_alternation.csv", sep = "\t")

# Check the structure of the data frame
head(genitive)
```

Describing categorical data

A categorical variable is made up of two or more discrete values. An intuitive way to describe categorical data would be to count how often each category occurs in the sample. These counts are then typically summarised in **frequency tables** and accompanied by suitable **graphs** (e.g., barplots).

Frequency tables (one variable)

Assume we are interested in how often each genitive variant (`"of"` vs. `"s"`) is attested in our data. In R, we can obtain their frequencies by inspecting the `Type` column of the `genitive` dataset. Since manual counting isn't really an option, we will make use of the convenient functions `table()` and `xtabs()`.

> The workhorse: `table()`
>
> This function requires a character vector. We use the notation `genitive$Type` to subset the `genitive` data frame according to the column `Type` (cf. data frames). We store the results in the variable `order_freq1` (you may choose a different name if you like) and display the output by applying to it the `print()` function.

```
# Count occurrences of genitive types ("s" and "of") in the data frame
gen_freq1 <- table(genitive$Type)

# Print table
print(gen_freq1)
```

```
  of    s
3103 1995
```

More detailed: `xtabs()`

Alternatively, you could use `xtabs()` to achieve the same result. The syntax is a little different, but it returns a slightly more more detailed table with explicit variable label(s).

```
# Count occurrences of genitive types ("s" and "of") in the data frame
gen_freq2 <- xtabs(~ Type, genitive)

# Print table
print(gen_freq2)
```

```
Type
  of    s
3103 1995
```

Frequency tables ($\geq$ 2 variables)

If we are interested in the relationship between **multiple categorical variables**, we can cross-tabulate the frequencies of their categories. For example, what is the distribution of clause order depending on the type of subordinate clause? The output is also referred to as a **contingency table**.

The `table()` way

```
# Get frequencies of genitive tpyes ("s" vs. "of") depending on the
 ↪  genre
gen_counts1 <- table(genitive$Type, genitive$Genre)

# Print contingency table
print(gen_counts1)
```

```
    Adventure Fiction General Fiction Learned Non-fiction Press
  of                408             425     587        1257   426
  s                 334             258     179         693   531
```

The `xtabs()` way

```
# Cross-tabulate Type and Genre
gen_counts2 <- xtabs(~ Type + Genre, genitive)

# Print cross-table
print(gen_counts2)
```

```
    Genre
Type Adventure Fiction General Fiction Learned Non-fiction Press
  of                408             425     587         1257   426
  s                 334             258     179          693   531
```

Percentage tables

There are several ways to compute percentages for your cross-tables, but by far the simplest is via the `prop.table()` function. As it only provides proportions, you can multiply the output by 100 to obtain percentages.

Get percentages for a `table()` object

```
# Convert to % using the prop.table() function
pct1 <- prop.table(gen_counts1, margin = 2) * 100

# Print percentages
print(pct1)
```

```
     Adventure Fiction General Fiction  Learned Non-fiction    Press
  of           54.98652        62.22548 76.63185    64.46154 44.51411
  s            45.01348        37.77452 23.36815    35.53846 55.48589
```

Get percentages for an `xtabs()` object

```
# Convert to % using the prop.table() function
pct2 <- prop.table(gen_counts2, margin = 2) * 100

# Print percentages
print(pct2)
```

```
    Genre
Type Adventure Fiction General Fiction  Learned Non-fiction    Press
  of           54.98652        62.22548 76.63185    64.46154 44.51411
```

```
s           45.01348        37.77452 23.36815     35.53846 55.48589
```

Notice how `pct2` still carries the variable labels `Genre` and `Type`, which is very convenient.

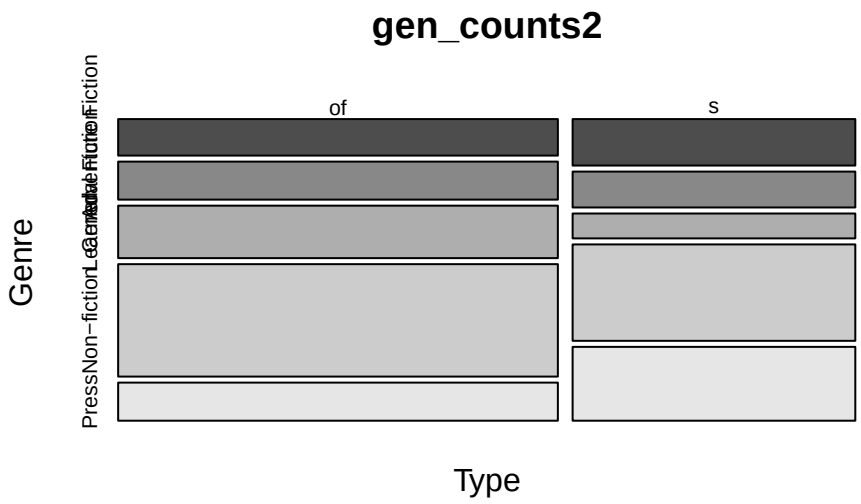
Plotting categorical data

This section demonstrates both the in-built plotting functions of R ('Base R') as well as the more modern versions provided by the `tidyverse` package.

Mosaicplots (raw counts)

A straightforward way to visualise a contingency table is the mosaicplot:

```
# Works with raw counts and percentages
# Using the output of xtabs() as input
mosaicplot(gen_counts2, color = TRUE)
```



Barplots (raw counts)

The workhorse of categorical data analysis is the barplot. Base R functions usually require a `table` object as input, whereas `ggplot2` can operate on the raw dataset.

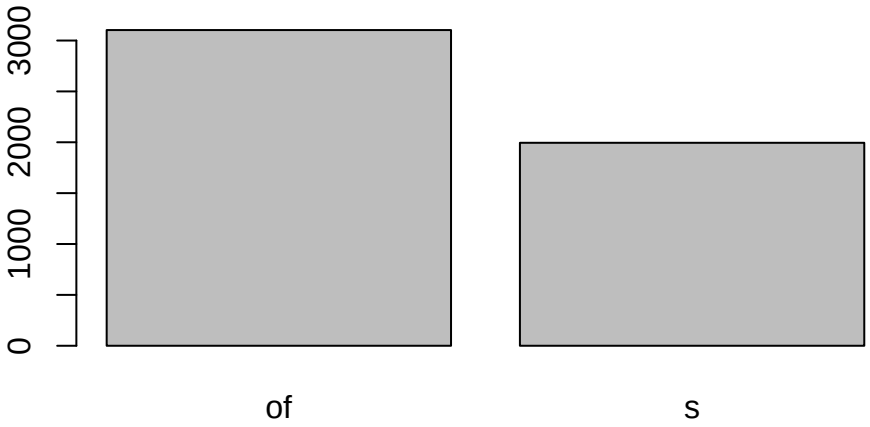
One variable

Base R

- Base R barplot with `barplot()`; requires the counts as computed by `tables()` or `xtabs()`

```r
# Generate cross-table
gen_freq1 <- table(genitive$Type)

# Create barplot
barplot(gen_freq1)
```

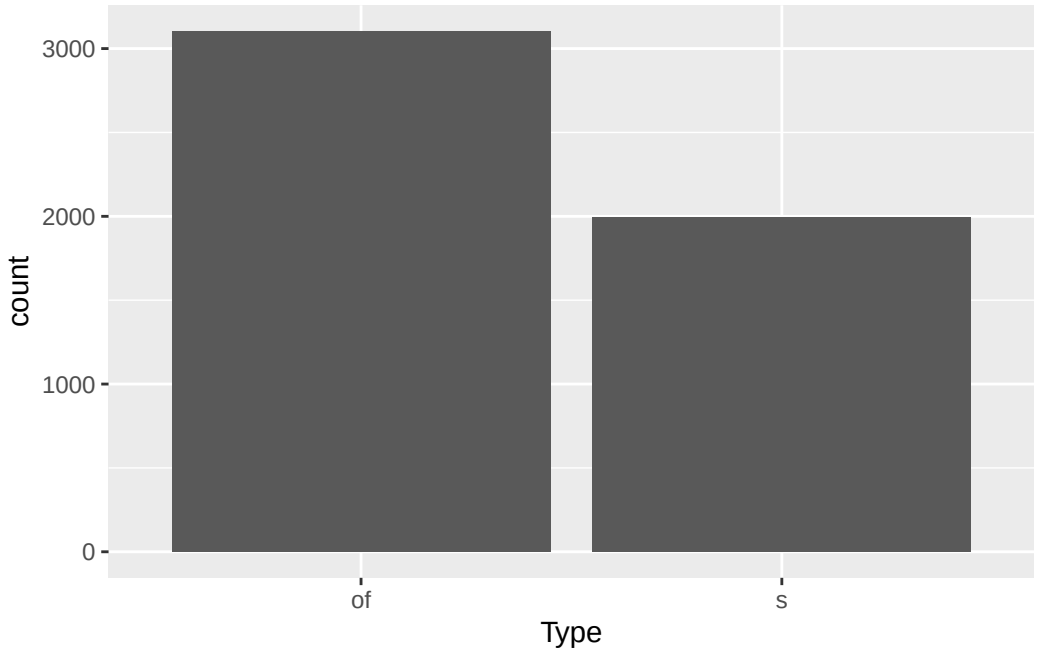


ggplot2

- Barplot with `geom_bar()` using the raw input data

```r
# Requirement: library(tidyverse)

# Create barplot
ggplot(genitive, aes(x = Type)) +
  geom_bar()
```



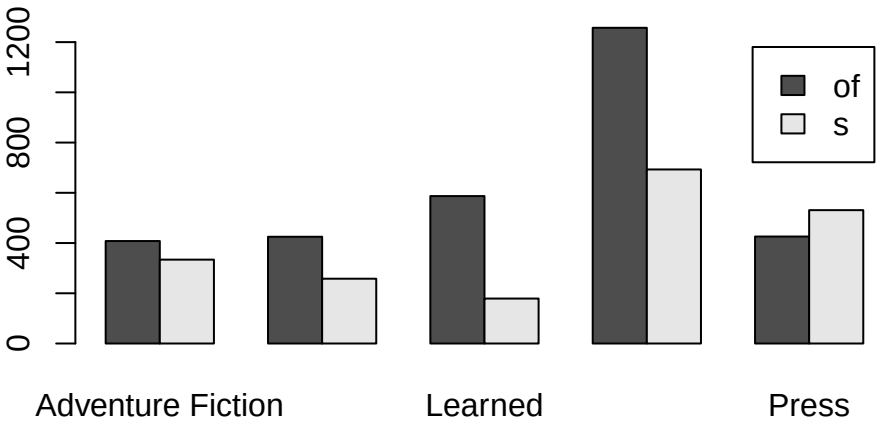
Two variables

Bivariate barplots can be obtained by either supplying a contingency table (Base R) or by mapping the second variable onto the `fill` argument using the raw data.

Base R

```
# Generate cross-table with two variables
gen_counts2 <- xtabs(~ Type + Genre, genitive)

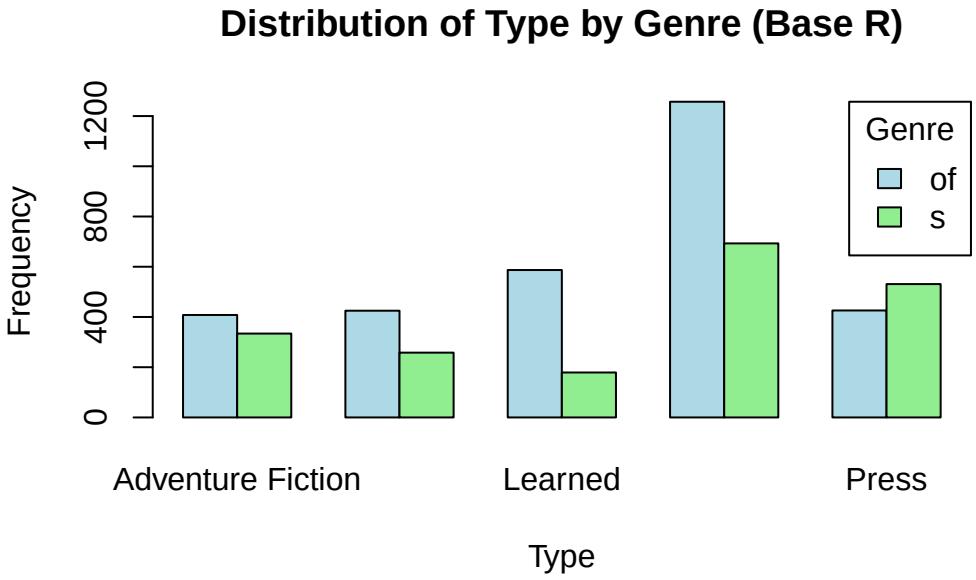
# Create simple barplot
barplot(gen_counts2,
        beside = TRUE,  # Make bars side-by-side
        legend = TRUE)  # Add a legend
```



Base R (fully customised)

```
# Generate cross-table with two variables
gen_counts2 <- xtabs(~ Type + Genre, genitive)

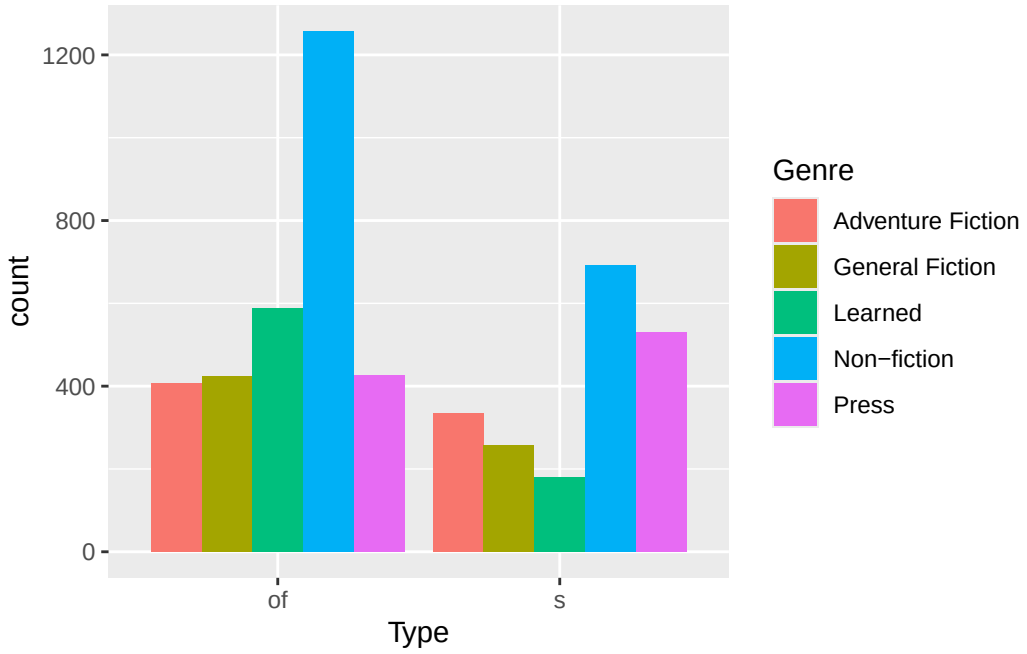
# Customise barplot with axis labels, colours and legend
barplot(gen_counts2,
        beside = TRUE,  # Make bars dodged (i.e., side by side)
        main = "Distribution of Type by Genre (Base R)",
        xlab = "Type",
        ylab = "Frequency",
        col = c("lightblue", "lightgreen"), # Customize colors
        legend = TRUE,  # Add a legend
        args.legend = list(title = "Genre", x = "topright"))
```



ggplot2

```
# Requirement: library(tidyverse)

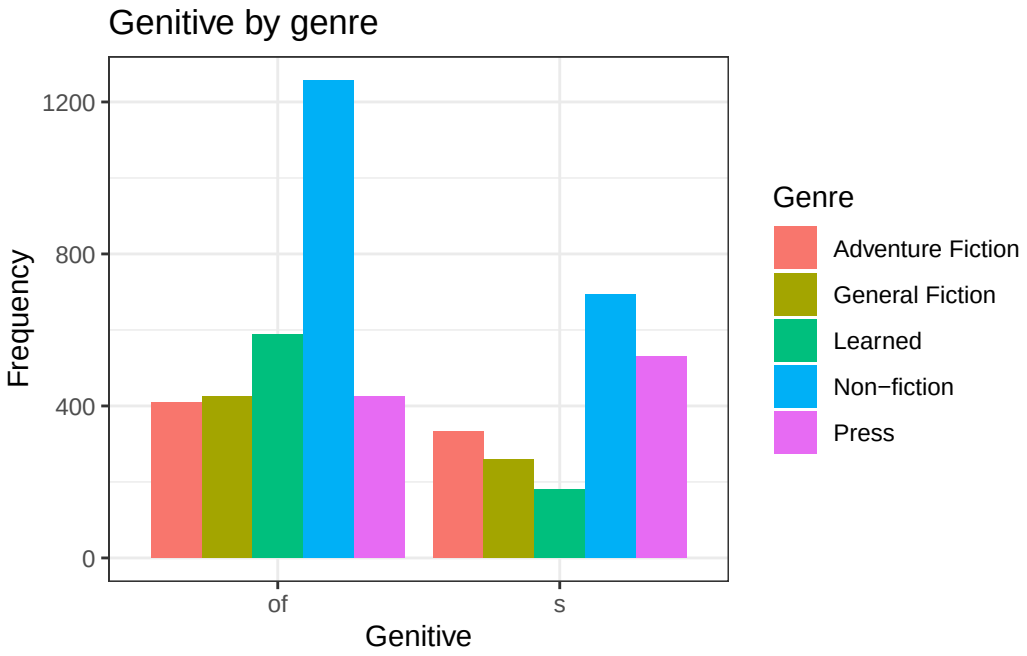
# Create simple barplot with the ggplot() function
ggplot(genitive, aes(x = Type, fill = Genre)) +
  geom_bar(position = "dodge")
```



ggplot2 (fully customised)

```
# Requirement: library(tidyverse)

# Fully customised ggplot2 object
ggplot(genitive, aes(x = Type, fill = Genre)) +
  geom_bar(position = "dodge") +
  labs(
    title = "Genitive by genre",
    x = "Genitive",
    y = "Frequency",
    fill = "Genre"
  ) +
  theme_bw()
```

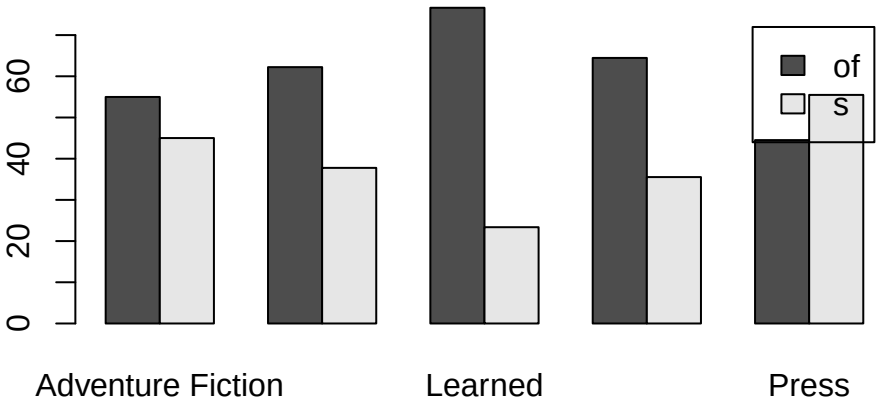


Barplots (percentages)

Base R

In very much the same way as with the raw counts:

```
# Create simple barplot with a percentage table as input
barplot(pct1,
        beside = TRUE,  # Make bars side-by-side
        legend = TRUE)  # Add a legend
```



ggplot2

Here, a few tweaks are necessary. Because the `ggplot()` function prefers to works with data frames rather than cross-tables, we'll have to coerce it into one first:

```
# Convert a percentage table to a data frame
# My recommendation: Use the pct2 object, which was generated using
 ↪  xtabs() because it will keep the variable names
pct2_df <- as.data.frame(pct2)

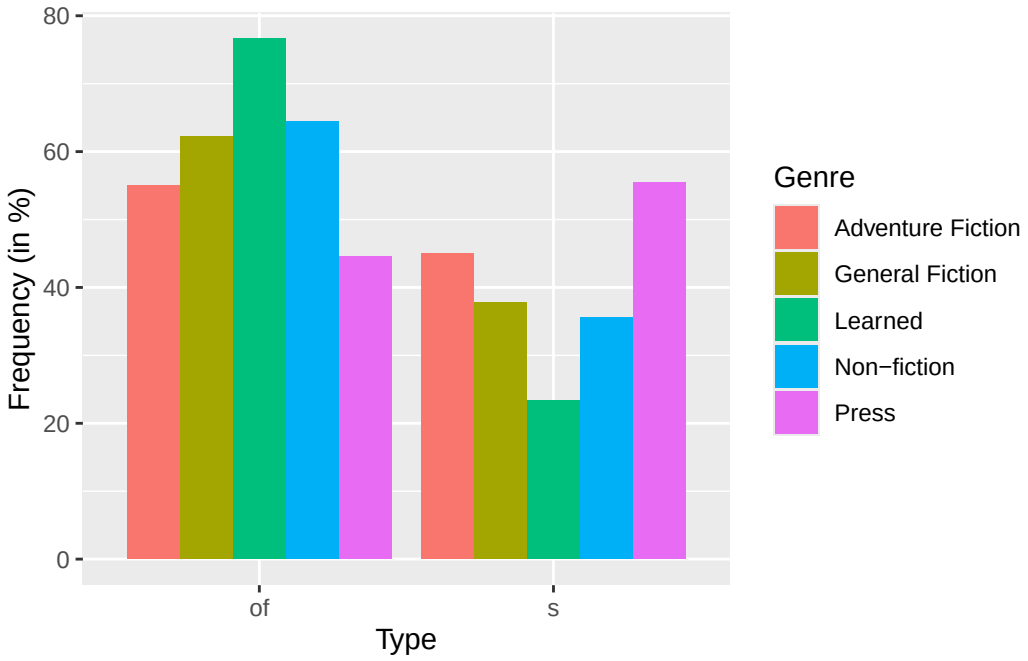
print(pct2_df)
```

```
   Type             Genre     Freq
1    of Adventure Fiction 54.98652
2     s Adventure Fiction 45.01348
3    of   General Fiction 62.22548
4     s   General Fiction 37.77452
5    of           Learned 76.63185
6     s           Learned 23.36815
7    of       Non-fiction 64.46154
8     s       Non-fiction 35.53846
9    of             Press 44.51411
10    s             Press 55.48589
```

Now we can plot the percentages with `geom_col()`. This geom (= 'geometric object') allows us to manually specify what should be mapped onto the y-axis:

```
# Requirement: library(tidyverse)

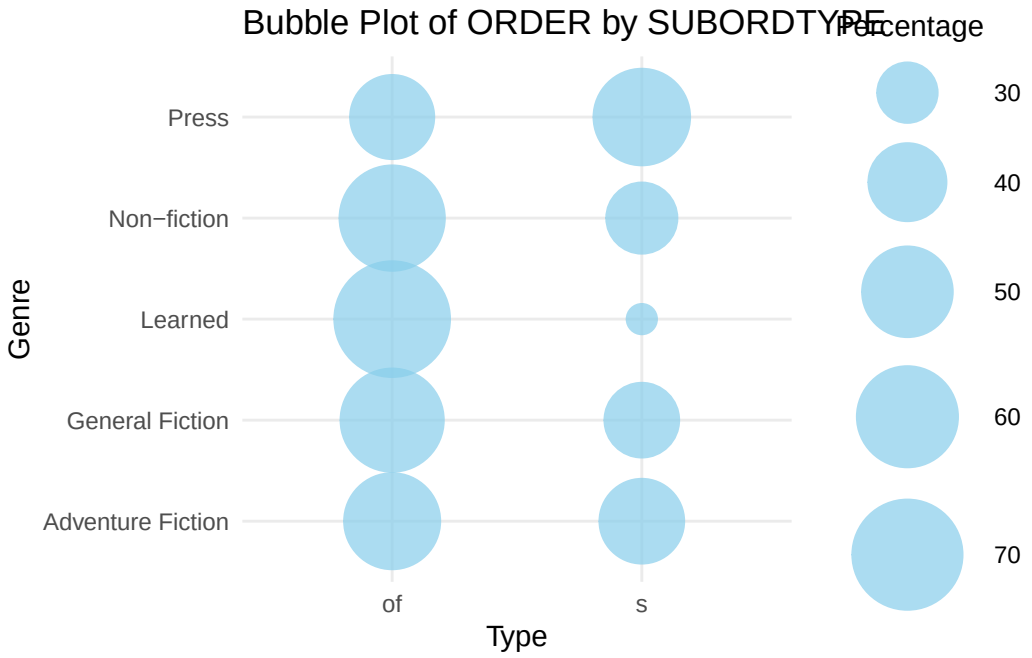
# Create barplot with user-defined y-axis, which requires geom_col()
 ↪  rather than geom_bar()
ggplot(pct2_df, aes(x = Type, y = Freq, fill = Genre)) +
  geom_col(position = "dodge") +
  labs(y = "Frequency (in %)")
```



Bubble plot (percentages)

```
# Requirement: library(tidyverse)

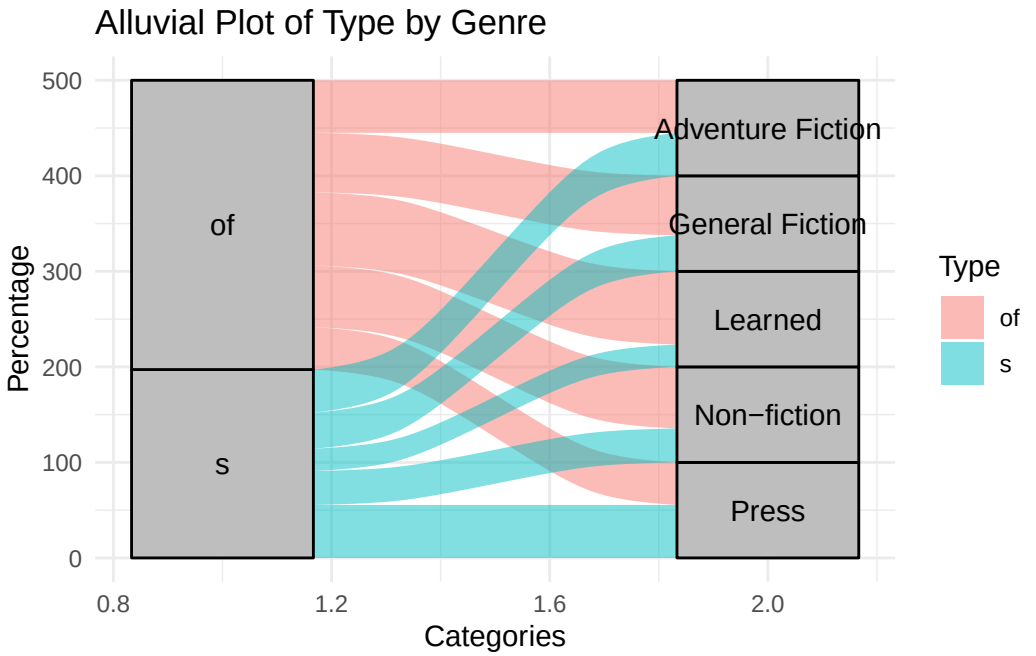
# Bubble plot
ggplot(pct2_df, aes(x = Type, y = Genre, size = Freq)) +
  geom_point(color = "skyblue", alpha = 0.7) +
  scale_size_continuous(range = c(5, 20)) +  # Adjust bubble size range
  labs(title = "Bubble Plot of ORDER by SUBORDTYPE",
       x = "Type",
       y = "Genre",
       size = "Percentage") +
  theme_minimal()
```



Alluvial plot (percentages)

```
# Make sure to install this library prior to running the code below
library(ggalluvial)

ggplot(pct2_df,
       aes(axis1 = Type, axis2 = Genre, y = Freq)) +
  geom_alluvium(aes(fill = Type)) +
  geom_stratum(fill = "gray") +
  geom_text(stat = "stratum", aes(label = after_stat(stratum))) +
  labs(title = "Alluvial Plot of Type by Genre",
       x = "Categories", y = "Percentage") +
  theme_minimal()
```



Exporting tables to MS Word

The `crosstable` and `flextable` packages make it very easy to export elegant tables to MS Word.

Clean and to the point: `crosstable()`

This is perhaps the most elegant solution. Generate a `crosstable()` object by supplying at the very least …

- the original dataset (`data = genitive`),
- the dependent variable (`cols = Type`), and
- the independent variable (`by = Genre`).

You can further specify …

- whether to include column totals, row totals or both (here: `total = both`),
- the rounding scheme (here: `percent_digits = 2`),
- …

```
# Required libraries:
# library(crosstable)
# library(flextable)

# Create the cross table
output1 <- crosstable(data = genitive,
                      cols = Type,
                      by = Genre,
                      total = "both",
                      percent_digits = 2)

# Generate file
as_flextable(output1)
```

label	variable	Genre				
		Adventure Fiction	General Fiction	Learned	Non-fiction	Press
Type	of	408 (13.15%)	425 (13.70%)	587 (18.92%)	1257 (40.51%)	426 (13.73%
	s	334 (16.74%)	258 (12.93%)	179 (8.97%)	693 (34.74%)	531 (26.62%
	Total	742 (14.55%)	683 (13.40%)	766 (15.03%)	1950 (38.25%)	957 (18.77%

How much info do you need? Yes.

It also possible to use `as_flextable()` without pre-processing the data with `crosstable()`; supplying a table preferably created with `xtabs()` is sufficient. Without any doubt, the output is extremely informative, yet it is everything but reader-friendly.
For this reason, I recommend relying on the less overwhelming `crosstable()` option above if a plain and easy result is desired. However, readers who would like to leverage the full capabilities of the `flextable()` package and familiarise themselves with the abundant options for customisation, can find the detailed documentation here.

```
# Requires the following library:
# library(flextable)

# Create a table
tab1 <- xtabs(~ Type + Genre, genitive)

# Directly convert a table to a flextable with as_flextable()
output_1 <- as_flextable(tab1)

# Print output
print(output_1)
```

		Genre					
Type		Adventure Fiction	General Fiction	Learned	Non-fiction	Press	Total
of	Count	408 (8.0%)	425 (8.3%)	587 (11.5%)	1,257 (24.7%)	426 (8.4%)	3,103 (60.9%)
	Mar. pct [1]	55.0% ; 13.1%	62.2% ; 13.7%	76.6% ; 18.9%	64.5% ; 40.5%	44.5% ; 13.7%	
s	Count	334 (6.6%)	258 (5.1%)	179 (3.5%)	693 (13.6%)	531 (10.4%)	1,995 (39.1%)
	Mar. pct	45.0% ; 16.7%	37.8% ; 12.9%	23.4% ; 9.0%	35.5% ; 34.7%	55.5% ; 26.6%	
Total	Count	742 (14.6%)	683 (13.4%)	766 (15.0%)	1,950 (38.3%)	957 (18.8%)	5,098 (100.0%)

Describing continuous data

Measures of central tendency

From here on out, we assume X is a continuous random variable with observations $\{x_1, x_2, ..., x_n\}$ and sample size n. Measures of central tendency offer convenient one-value-summaries of the distribution of X as well as estimations of population parameters, granted some corrective steps.

The sample mean

The population mean μ can be approximated rather well by the sample mean

$$\hat{\mu} = \frac{x_1 + x_2 + ... + x_n}{n} = \frac{1}{n}\sum_{i=1}^{n} x_i. \tag{1}$$

In R, we can obtain the average value of a numeric vector with the `mean()` function.

In the `genitive` data, "the type–token ratio (TTR) over the five sentences preceding and following each token was calculated" (Grafmiller 2014: 479). Thus, on average, each sentence has a lexical density of

```
mean(genitive$Type_Token_Ratio)
```

```
[1] 74.74926
```

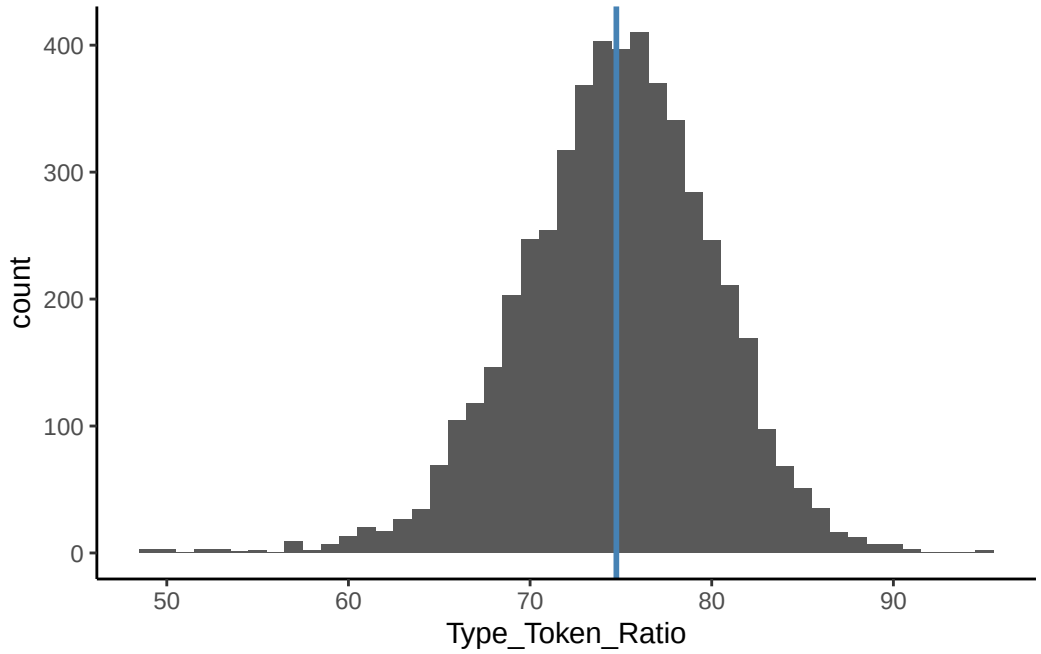
The output returned by this function provides a one-value summary of all observations contained in `Type_Token_Ratio`. Because the the mean $\bar{x}$ takes into account all data points, it is prone to the influence of **outliers**, i.e., extreme values.

The distribution of continuous variables is best visualised in terms of histograms or density plots, which are illustrated for `Type_Token_Ratio`. The blue line indicates the sample mean.

Histogram (ggplot2)

```
# Plot distribution of Type_Token_Ratio
gen_hist <- ggplot(genitive, aes(x = Type_Token_Ratio)) +
                  geom_histogram(binwidth = 1)

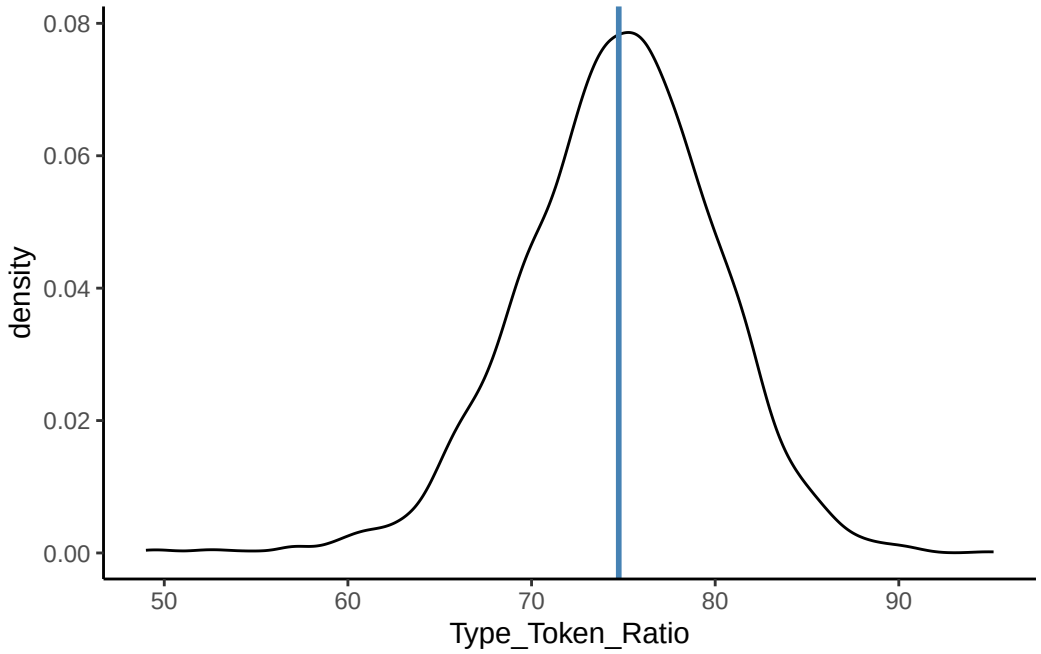
gen_hist +
  # Add mean
  geom_vline(aes(xintercept = mean(Type_Token_Ratio)),
             color = "steelblue",
             linewidth = 1) +
  theme_classic()
```



Density plot (ggplot2)

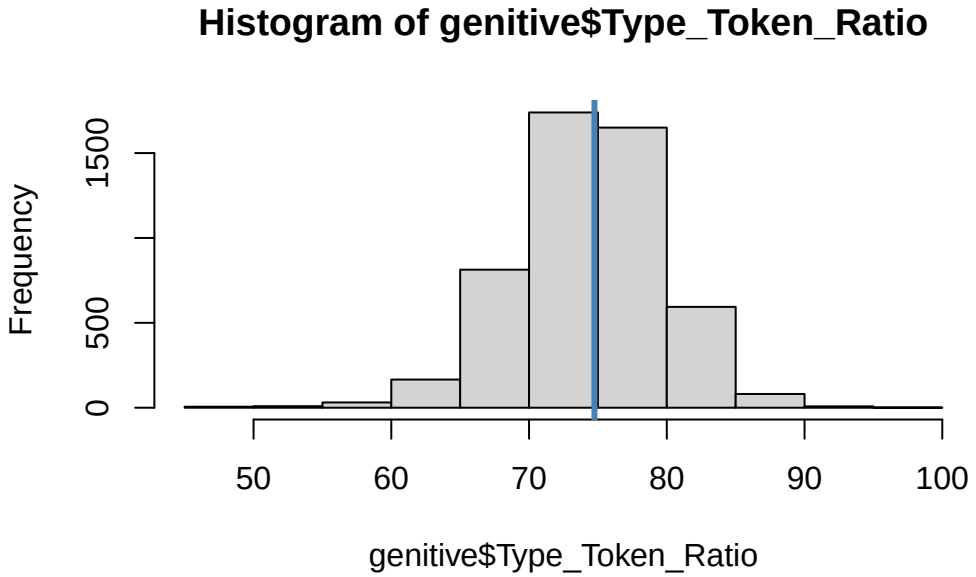
```
# Plot distribution of Type_Token_Ratio
gen_dens <- ggplot(genitive, aes(x = Type_Token_Ratio)) +
                  geom_density()

gen_dens +
  # Add mean
  geom_vline(aes(xintercept = mean(Type_Token_Ratio)),
             color = "steelblue",
             linewidth = 1) +
  theme_classic()
```



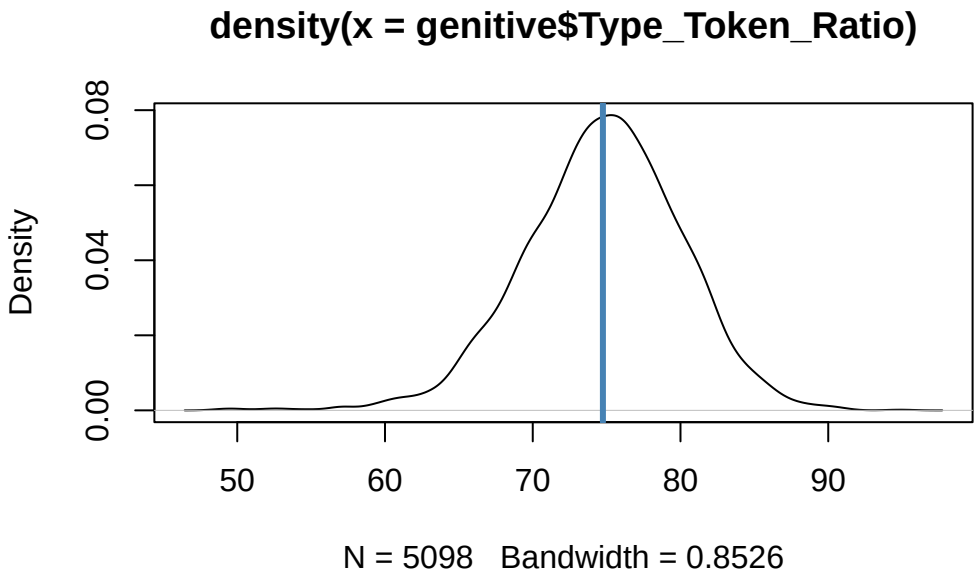
Histogram (Base R)

```
hist(genitive$Type_Token_Ratio)
  abline(v=mean(genitive$Type_Token_Ratio),lwd=3, col = "steelblue")
```



Density plot (Base R)

```
plot(density(genitive$Type_Token_Ratio))
  abline(v=mean(genitive$Type_Token_Ratio),lwd=3, col = "steelblue")
```



The median

The `median()` function computes the "the halfway point of the data (50% of the data are above the median; 50% of the data are below" (Winter 2020: 58). As such, it is the measure of choice for data with many outliers as well as for ordinal data (e.g. Likert-scale ratings).

$$\tilde{x}_{0.5} = \begin{cases} x_{((n+1)/2)} & \text{if } n \text{ is odd.} \\ \frac{1}{2}(x_{n/2} + x_{(n/2+1)}) & \text{if } n \text{ is even.} \end{cases} \quad (2)$$

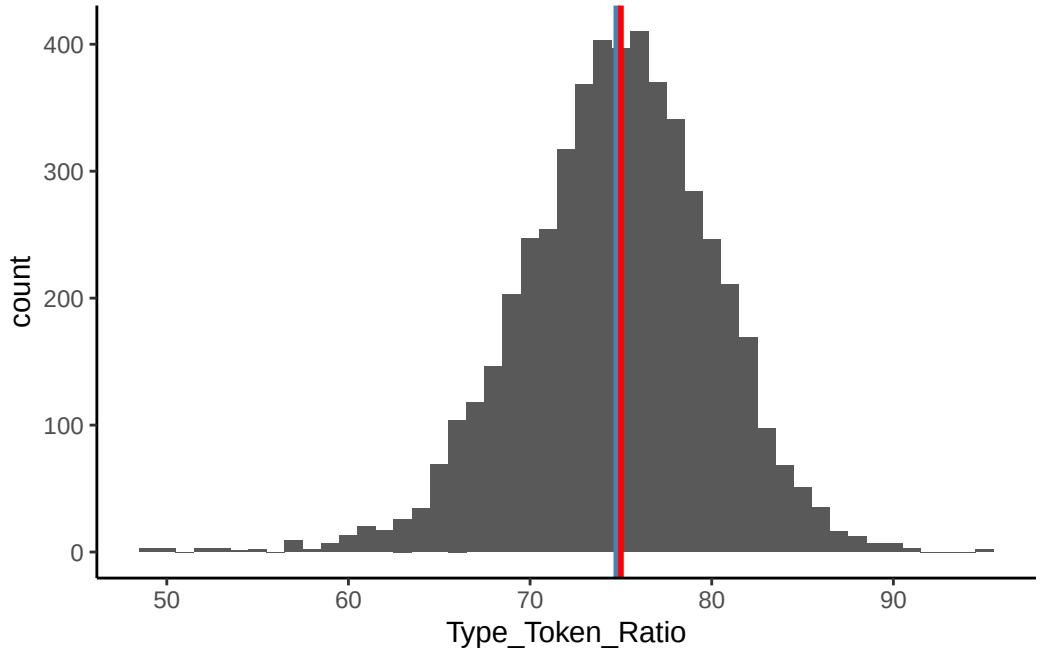
```
median(genitive$Type_Token_Ratio)
```

```
[1] 75
```

The median of `Type_Token_Ratio` is represented by the red vertical line.

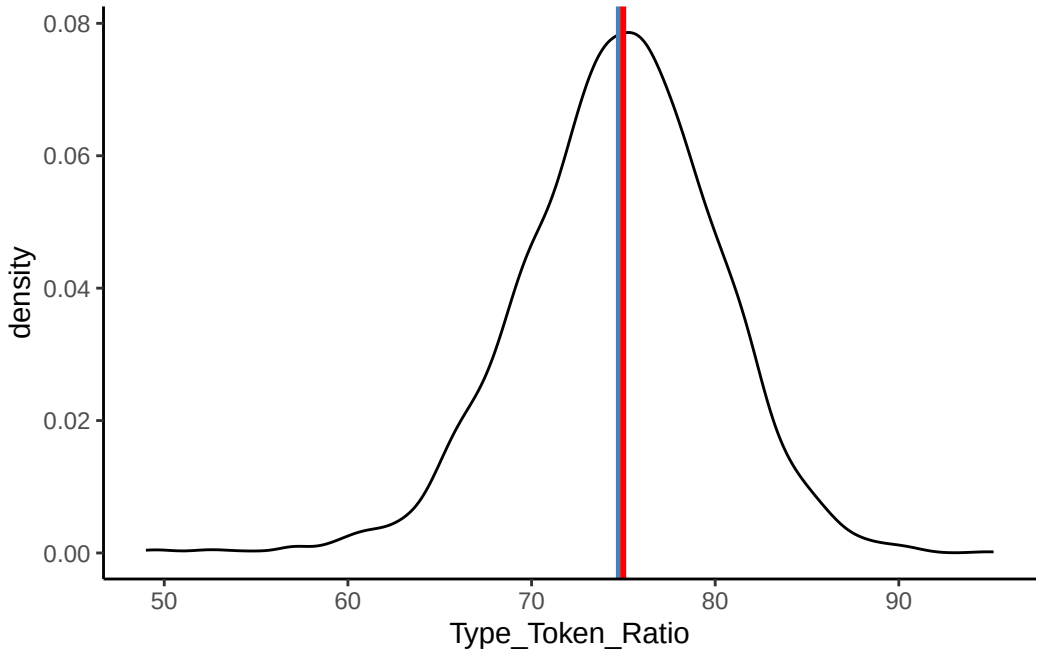
Histogram (ggplot2)

```
gen_hist +
  # Add mean
  geom_vline(aes(xintercept = mean(Type_Token_Ratio)), color = "steelblue",
   ↪  linewidth = 1) +
  # Add median
  geom_vline(aes(xintercept = median(Type_Token_Ratio)), color = "red",
   ↪  linewidth = 1) +
  theme_classic()
```



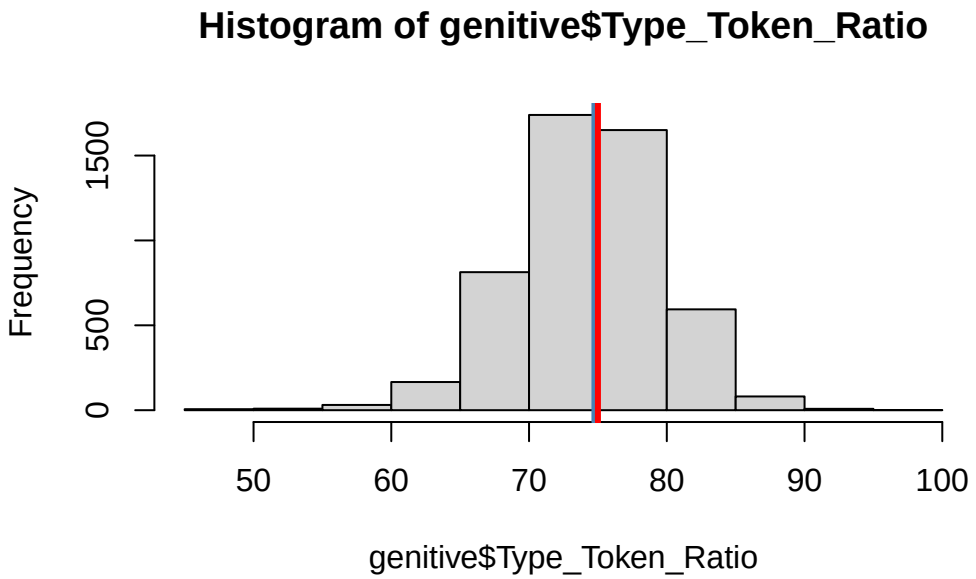
Density plot (ggplot2)

```
gen_dens +
  # Add mean
  geom_vline(aes(xintercept = mean(Type_Token_Ratio)), color = "steelblue",
   ↪  linewidth = 1) +
  # Add median
  geom_vline(aes(xintercept = median(Type_Token_Ratio)), color = "red",
   ↪  linewidth = 1) +
  theme_classic()
```



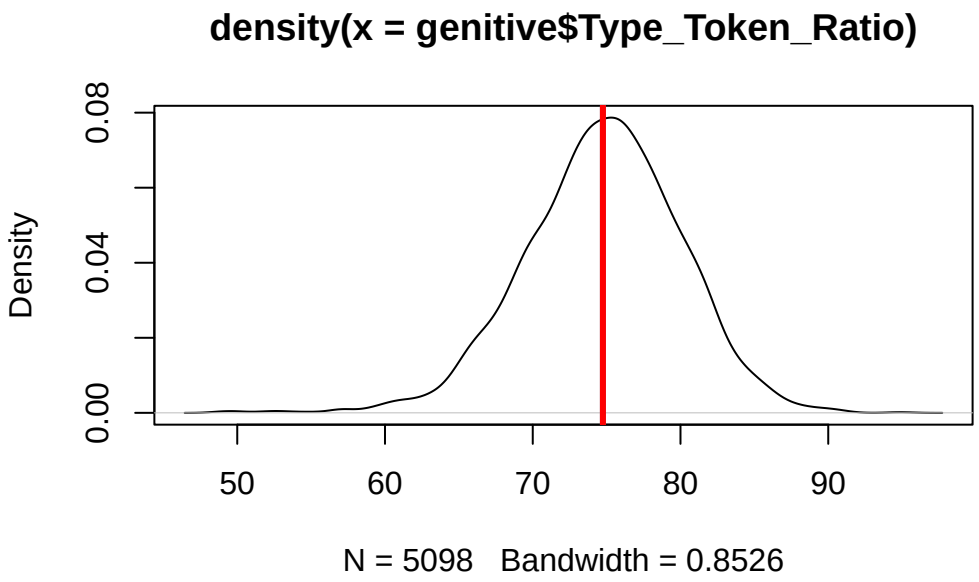
Histogram (Base R)

```
hist(genitive$Type_Token_Ratio)
  abline(v=mean(genitive$Type_Token_Ratio),lwd=3, col = "steelblue")
  abline(v=median(genitive$Type_Token_Ratio),lwd=3, col = "red")
```



Density plot (Base R)

```
plot(density(genitive$Type_Token_Ratio))
  abline(v=mean(genitive$Type_Token_Ratio),lwd=3, col = "steelblue")
  abline(v=mean(genitive$Type_Token_Ratio),lwd=3, col = "red")
```



Sample variance and standard deviation

In order to assess how well the mean represents the data, it is instructive to compute the **variance** `var()` and the **standard deviation** `sd()` for a sample.

The unbiased estimator of the population variance σ^2 is defined as

$$\hat{\sigma}^2 = \frac{\sum_{i=1}^{n}(x_i - \hat{\mu})^2}{n-1}. \tag{3}$$

In other words, it represents the average squared deviation of all observations from the sample mean.

```
var(genitive$Type_Token_Ratio)
```

```
[1] 28.34916
```

Correspondingly, the standard deviation σ of the population mean can be estimated via the square root of the sample variance:

$$\hat{\sigma} = \sqrt{\hat{\sigma}^2} = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \hat{\mu})^2}{n-1}} \tag{4}$$

```
sd(genitive$Type_Token_Ratio)
```

```
[1] 5.324393
```

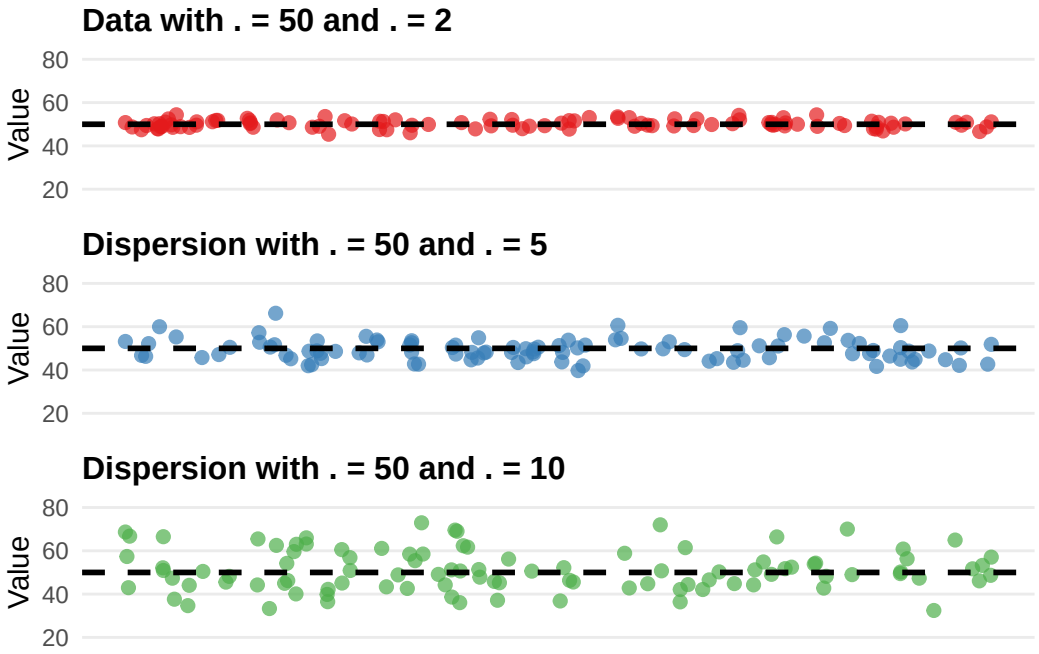
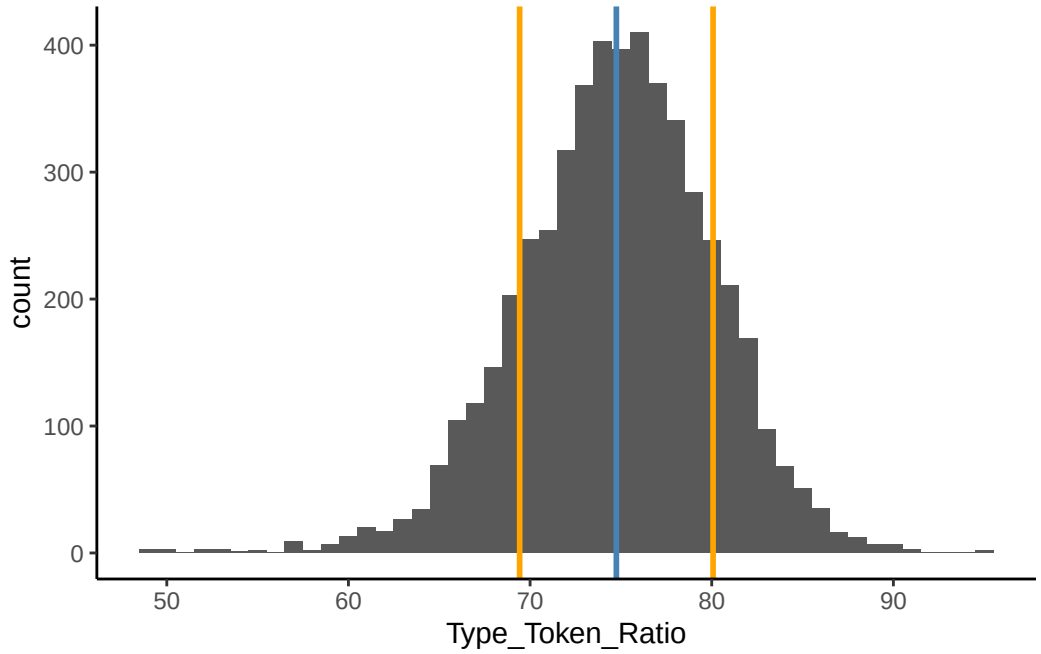


Figure 1: Comparison of different parameter values

Example 1

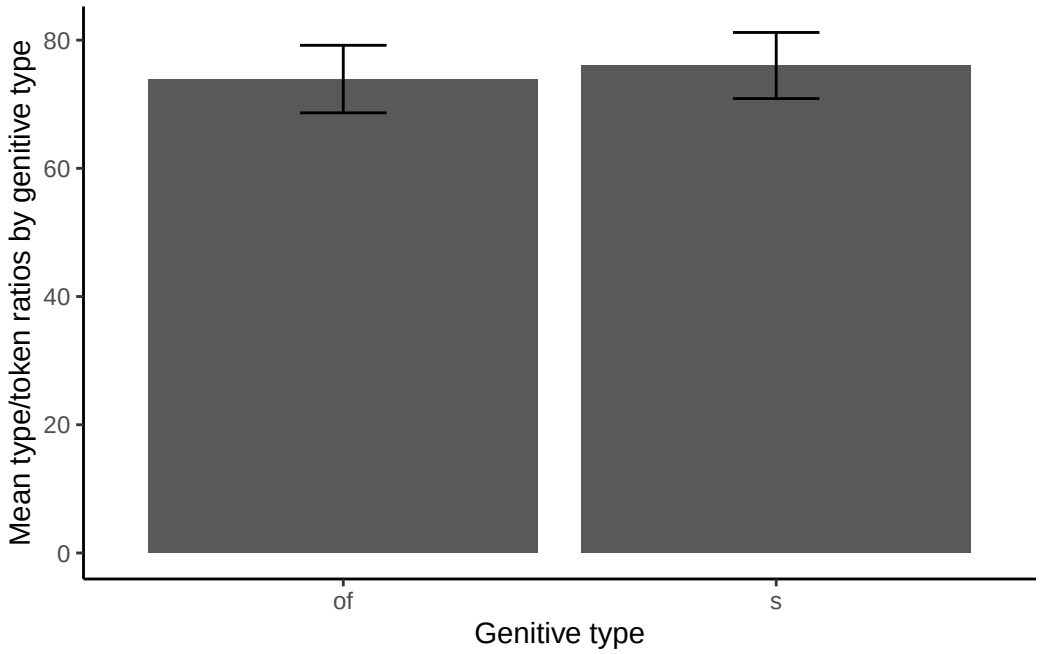
```
gen_hist +
  # Add verticle line for the mean
  geom_vline(aes(xintercept = mean(Type_Token_Ratio)), color = "steelblue",
   ↪  linewidth = 1) +
  # Add -1sd
  geom_vline(aes(xintercept = mean(Type_Token_Ratio) -
   ↪  sd(Type_Token_Ratio)), color = "orange", linewidth = 1) +
  # Add +1sd
  geom_vline(aes(xintercept = mean(Type_Token_Ratio) +
   ↪  sd(Type_Token_Ratio)), color = "orange", linewidth = 1) +
  theme_classic()
```



Example 2

```
# Create data frame with mean and sd for each TYPE
gen_mean_sd <- genitive %>%
  # Select variables of interest
  select(Type, Type_Token_Ratio) %>%
  # Group results of following operations by TYPE
  group_by(Type) %>%
    # Create grouped summary of mean and sd for each TYPE
    summarise(mean = mean(Type_Token_Ratio),
                sd = sd(Type_Token_Ratio))

# Plot results
ggplot(gen_mean_sd, aes(x = Type, y = mean)) +
  # Barplot with a specific variable mapped onto y-axis
  geom_col() +
  # Add mean and standard deviation to the plot
  geom_errorbar(aes(x = Type,
                    ymin = mean-sd,
                    ymax = mean+sd), width = .2) +
  theme_classic() +
  labs(y = "Mean type/token ratios by genitive type", x = "Genitive type")
```



Quantiles

While `median()` divides the data into two equal sets (i.e., two 50% quantiles), the `quantile()` function makes it possible to partition the data further.

```
quantile(genitive$Type_Token_Ratio)
```

```
      0%      25%      50%      75%     100%
49.00000 71.00000 75.00000 78.00000 95.16129
```

`quantile(x, 0)` and `quantile(x, 1)` thus show the minimum and maximum values, respectively.

```
quantile(genitive$Type_Token_Ratio, 0)
```

```
0%
49
```

```
quantile(genitive$Type_Token_Ratio, 1)
```

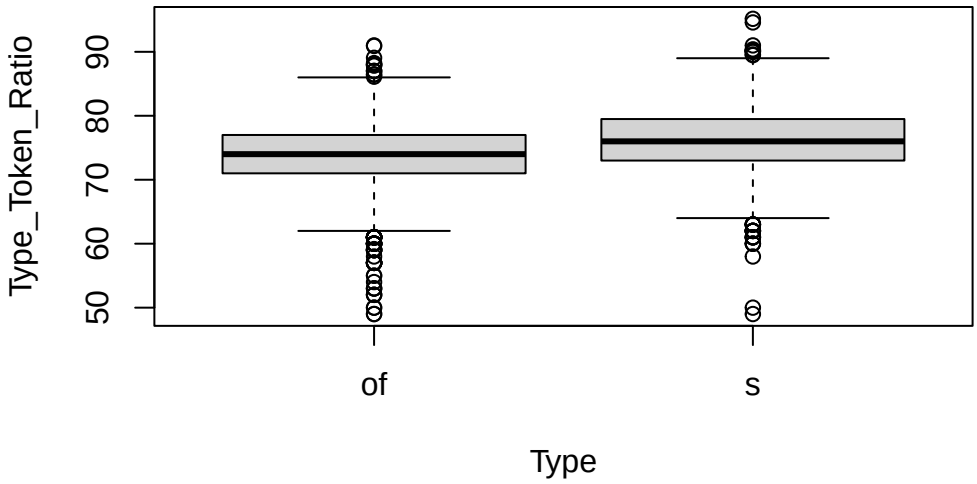
```
    100%
95.16129
```

Quartiles and boxplots

For each genitive variant, the distribution of type/token ratios across the dataset is visualised in the boxplots below. The thick horizontal line within each box represents the median of the distribution. The box itself spans the **interquartile range (IQR)**, extending from the 25th to the 75th percentile. Data points that lie more than 1.5 times the IQR above or below the box are classified as **outliers** and are shown as individual dots.

Boxplot (Base R)

```
boxplot(Type_Token_Ratio ~ Type, genitive)
```



Tip: You can extract the outliers from the boxplot and match them to the original rows in the dataset as follows:

```
# Save the boxplot object
bp <- boxplot(Type_Token_Ratio ~ Type, genitive)
```

```
# Extract the outlier values
outlier_values <- bp$out

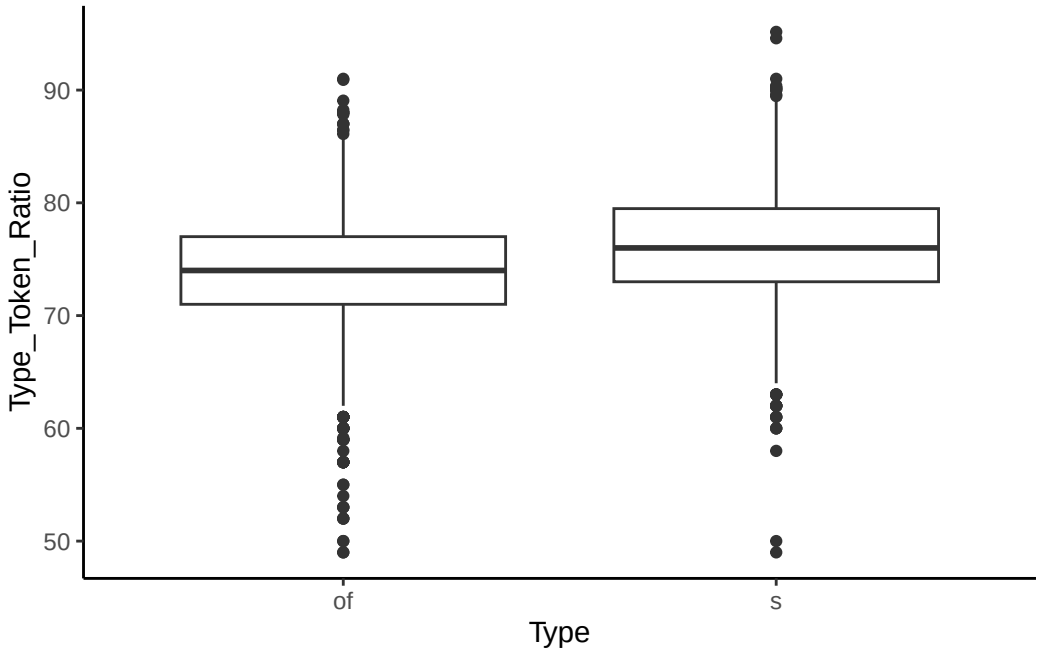
# View them
print(outlier_values)

# Match to the original rows in the data
outliers_df <- genitive[genitive$Type_Token_Ratio %in% outlier_values, ]

# Show the rows
print(outliers_df)
```

Boxplot (ggplot2)

```
ggplot(genitive, aes(x = Type, y = Type_Token_Ratio)) +
  geom_boxplot() +
  theme_classic()
```



Bivariate statistics

Covariance

Covariance "measures the average tendency of two variables to covary (change together)"(Baguley 2012: 206). Recall the variance estimator from Equation 3, which has the expanded form

$$\hat{\sigma}^2 = \frac{\sum_{i=1}^{n}(x_i - \hat{\mu})(x_i - \hat{\mu})}{n-1}.$$

```
var(genitive$Possessor_Length)
```

```
[1] 1.944548
```

The **covariance** is obtained by replacing one of the product terms with another variable Y, i.e.,

$$\hat{\sigma}_{X,Y} = \frac{\sum_{i=1}^{n}(x_i - \hat{\mu}_X)(y_i - \hat{\mu}_Y)}{n-1}. \tag{5}$$

The covariance of `Possessor_Length` (***John's*** *cat*) and `Possessum_Length` (*John's* ***cat***) is negligible:

```
cov(genitive$Possessor_Length, genitive$Possessum_Length)
```

```
[1] 0.1693295
```

Correlation

Covariance is typically used as an intermediary measure for the calculation of the **correlation coefficient** r (or ρ, also known as Pearson's product-moment correlation coefficient), which involves dividing the covariance by the product of the standard deviations of X and Y:

$$r_{X,Y} = \frac{\hat{\sigma}_{X,Y}}{\hat{\sigma}_X \hat{\sigma}_Y} \tag{6}$$

This returns a measure in the interval $[-1, 1]$, with

- $0 < r \leq 1$ suggesting a positive correlation (increasing X-values $\sim$ increasing Y-values) and
- $-1 \leq r < 0$ a negative correlation (increasing X-values $\sim$ decreasing Y-values). It is best thought of as the extent to which two variables form a straight-line (linear) relationship.

For example, the `cor()` function shows that the length of the possessor is weakly yet positively correlated with the length of the possessum.

```
cor(genitive$Possessor_Length, genitive$Possessum_Length)
```

```
[1] 0.1021913
```

Its squared version r^2 (or R^2) is known as the coefficient of determination and denotes "the proportion of variance in Y accounted for by X (or vice versa)" (Baguley 2012: 209). It turns out that possessor length only explains approximately 1% of the variance in possessum length:

```
cor(genitive$Possessor_Length, genitive$Possessum_Length)^2
```

```
[1] 0.01044307
```

Exercises

> Solutions
>
> You can find the solutions to the exercises here.

Tier 1

Exercise 1. Load the dataset `psycho_data` which contains several distributional and psycholinguistic measurements for 407 verbs.

```
library(readxl)
psycho_data <- read_xlsx("psycholing_data.xlsx")
```

Word frequencies follow a very characteristic distribution. Create a histogram of `Frequency` and characterise its distribution using the sample mean and median.

Exercise 2. How does the overall distribution as well as the position of the mean/median change if the frequency counts are log-transformed (cf. the `Log_frequency` column)? Why do log-transformations have this effect?

Exercise 3. Plot the verbs' concreteness ratings (1 = abstract, 5 = concrete) against their number of meanings using a scatterplot (`geom_point()`), and calculate Pearson's r and r^2. Briefly describe the relationship between `Concreteness` and `Number_meanings`.

Tier 2

Exercise 4. Plot the following variables from the genitive data and characterise the figures briefly:

- `Type` and `Possessor_Animacy2` (in %)
- `Type` and `Possessor_Givenness` (in %)
- `Type` and `Possessor_NP_Type` (in %)

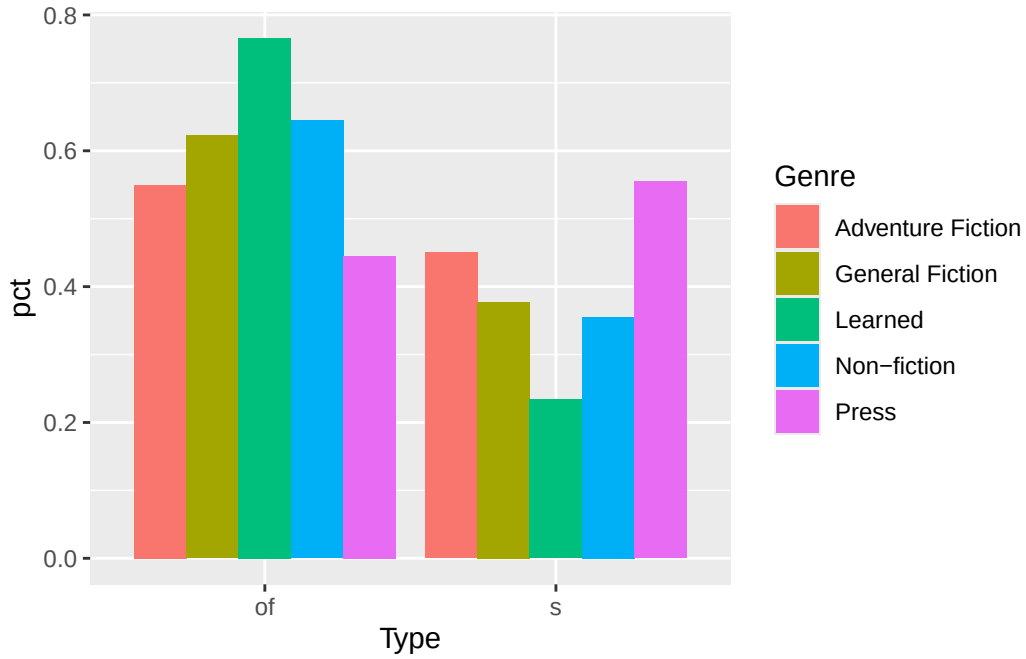
Hint: The `tidyverse` syntax offers numerous convenient functions to handle common data analysis tasks in an elegant fashion. For instance, a pipeline for computing frequencies and percentages for `Type` by `Genre` could look like this:

```
type_genre <- genitive %>%
                  group_by(Genre) %>%
                  count(Type) %>%
                  mutate(pct = n/sum(n))

print(type_genre)
```

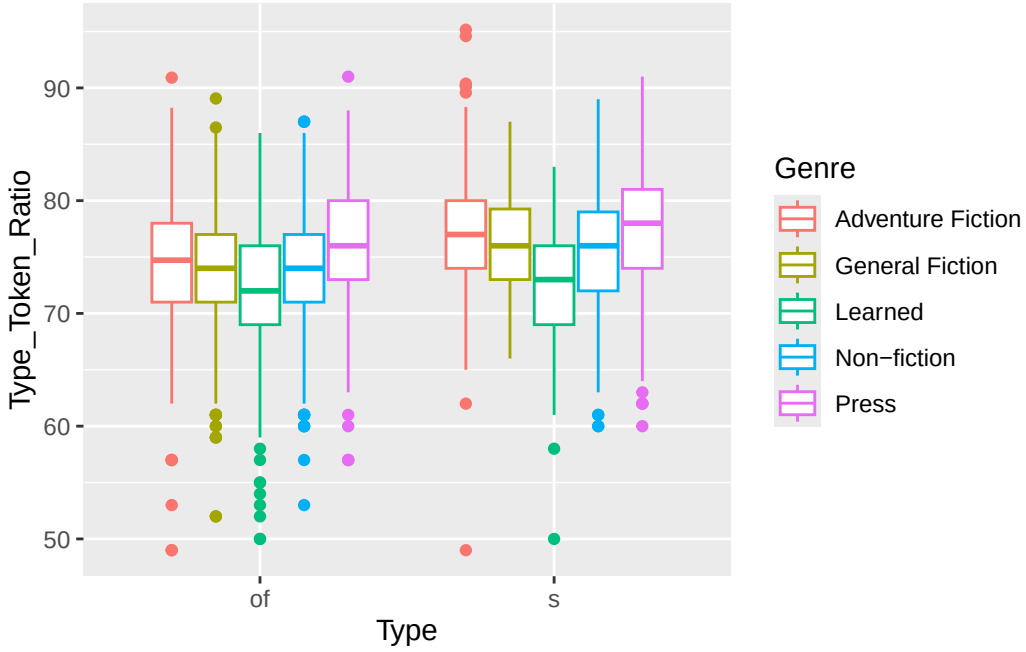
```
# A tibble: 10 x 4
# Groups:   Genre [5]
   Genre             Type      n   pct
   <chr>             <chr> <int> <dbl>
 1 Adventure Fiction of      408 0.550
 2 Adventure Fiction s       334 0.450
 3 General Fiction   of      425 0.622
 4 General Fiction   s       258 0.378
 5 Learned           of      587 0.766
 6 Learned           s       179 0.234
 7 Non-fiction       of     1257 0.645
 8 Non-fiction       s       693 0.355
 9 Press             of      426 0.445
10 Press             s       531 0.555
```

```
type_genre %>%
  ggplot(aes(x = Type, y = pct, fill = Genre)) +
  geom_col(pos = "dodge")
```



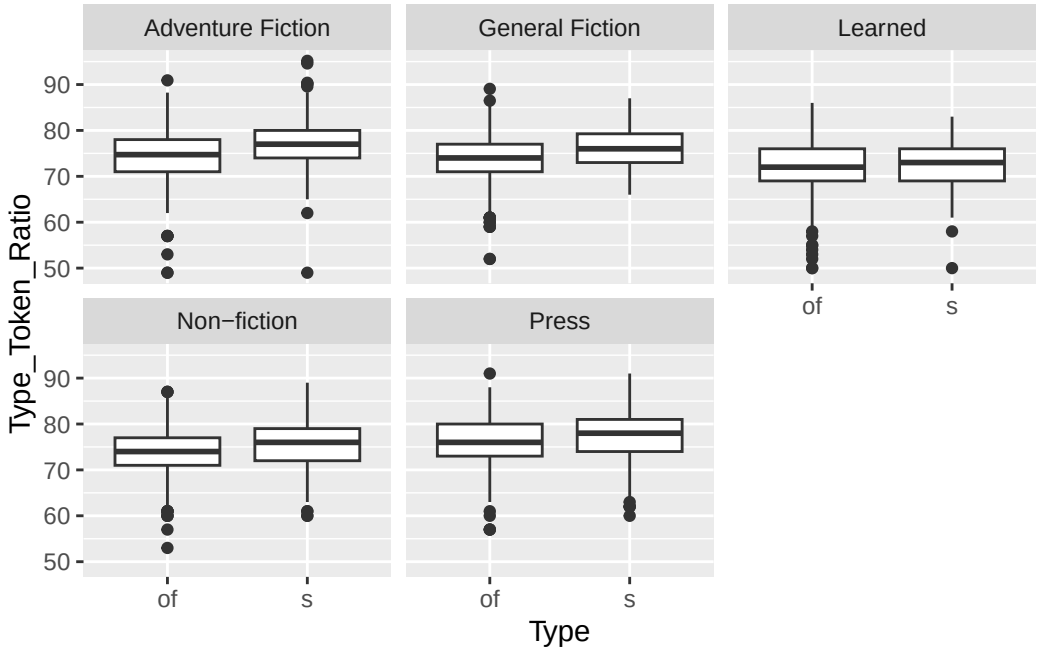
Exercise 5. Nearly all plotting functions that use the `ggplot()` graphics engine support positional arguments for more than two variables, rendering it an attractive option for multivariate plots. For instance, the distribution of `Type` by `Type_Token_Ratio` and `Genre` can be visualised by mapping `Genre` onto the `col` argument:

```
genitive %>%
  ggplot(aes(x = Type, y = Type_Token_Ratio, col = Genre)) +
  geom_boxplot()
```



Alternatively, for a slightly less cluttered representation, distinct subplots can be generated with `facet_wrap(~Variable)`:

```
genitive %>%
  ggplot(aes(x = Type, y = Type_Token_Ratio)) +
  geom_boxplot() +
  facet_wrap(~Genre)
```



Visualise the percentage of genitive types by `Possessum_Animacy2` in each `Genre`! Provide a short assessment of the results.

Exercise 6. The grouping function `group_by()` from the `tidyverse` library allows performing statistical operations on a per-group basis. These are typically followed by `summarise()`. For instance, computing the mean type/token ratio **for every genre** could be achieved using the following syntax:

```
genre_ttr <- genitive %>%
               group_by(Genre) %>%
               summarise(mean_TTR = mean(Type_Token_Ratio))

print(genre_ttr)
```

```
# A tibble: 5 x 2
  Genre              mean_TTR
  <chr>                 <dbl>
1 Adventure Fiction      75.6
```

```
2 General Fiction       75.0
3 Learned               72.2
4 Non-fiction           74.4
5 Press                 76.7
```

Extend the above code to include the standard deviation of the mean type/token ratio for every genre. Based on this updated data frame, generate a suitable barplot with error bars that represent the standard deviation of the mean.

Tier 3

Exercise 7. The standard error of the mean ($\hat{\sigma}_{\hat{\mu}}$) tells us how precisely we know the population mean μ based on our sample. It is calculated as

$$\hat{\sigma}_{\hat{\mu}} = \frac{\hat{\sigma}}{\sqrt{n}}, \tag{7}$$

where $\hat{\sigma}$ is the sample standard deviation and n is the sample size.

Calculate and interpret the standard error for `Type_Token_Ratio` by genitive `Type` and `Genre`.

Exercise 8. Standard errors can be used to construct confidence intervals (CIs) for a parameter estimate $\hat{\theta}$ (e.g., the sample mean). They have the general form:

$$\hat{\theta} \pm \text{Margin of Error}.$$

If the sample variance is known, the CIs can be estimated from a normal distribution as follows:

$$\hat{\mu} \pm z_{1-\alpha/2} \times \hat{\sigma}_{\hat{\mu}}. \tag{8}$$

As Baguley (2012: 79) explains, "$\hat{\mu}$ is the usual sample estimate of the arithmetic mean, $z_{1-\alpha/2}$ is the required quantile of the standard normal distribution and $\hat{\sigma}_{\hat{\mu}}$ is the sample estimate of the standard error of the mean".

Example: Given $n = 50, \hat{\mu} = 94.6, \hat{\sigma} = 19.6$ and a confidence level $\alpha = 0.05$, the 95% CI would be [89.17, 100.03].

```
# Mean - Margin of Error * SD / sqrt(N)
94.6 - qnorm(1-0.05 / 2) * 19.6 / sqrt(50)
```

```
[1] 89.16726
```

```
# Mean + Margin of Error * SD / sqrt(N)
94.6 + qnorm(1-0.05 / 2) * 19.6 / sqrt(50)
```

```
[1] 100.0327
```

Calculate the mean, standard deviation, standard error of the mean and the 95% confidence intervals of the mean for the `Possessor_Length` of each genitive `Type` in the `genitive` data! How do these measures contribute to our understanding of the genitive alternation?

References

Baguley, Thomas. 2012. *Serious Stats: A Guide to Advanced Statistics for the Behavioral Sciences*. Houndmills, Basingstoke: Palgrave Macmillan.

Grafmiller, Jason. 2014. "Variation in English Genitives Across Modality and Genres." *English Language and Linguistics* 18 (3): 471–96.

———. 2023. "The genitive alternation in 1960s and 1990s American English: Data from the Brown and Frown corpora." DataverseNO. https://doi.org/10.18710/R7HM8J.

Heumann, Christian, Michael Schomaker, and Shalabh. 2022. *Introduction to Statistics and Data Analysis: With Exercises, Solutions and Applications in r*. 2nd ed. Cham: Springer. https://doi.org/10.1007/978-3-031-11833-3.

Wickham, Hadley, Mine Çetinkaya-Rundel, and Garrett Grolemund. 2023. *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. Beijing: O'Reilly. https://r4ds.hadley.nz.

Winter, Bodo. 2020. *Statistics for Linguists: An Introduction Using r*. New York; London: Routledge.